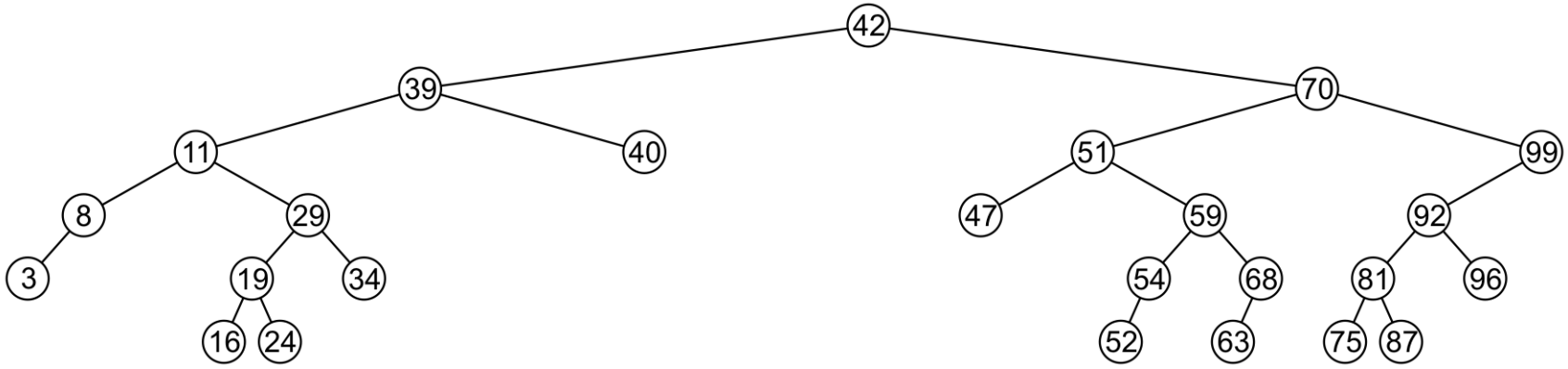
Deleting a Node

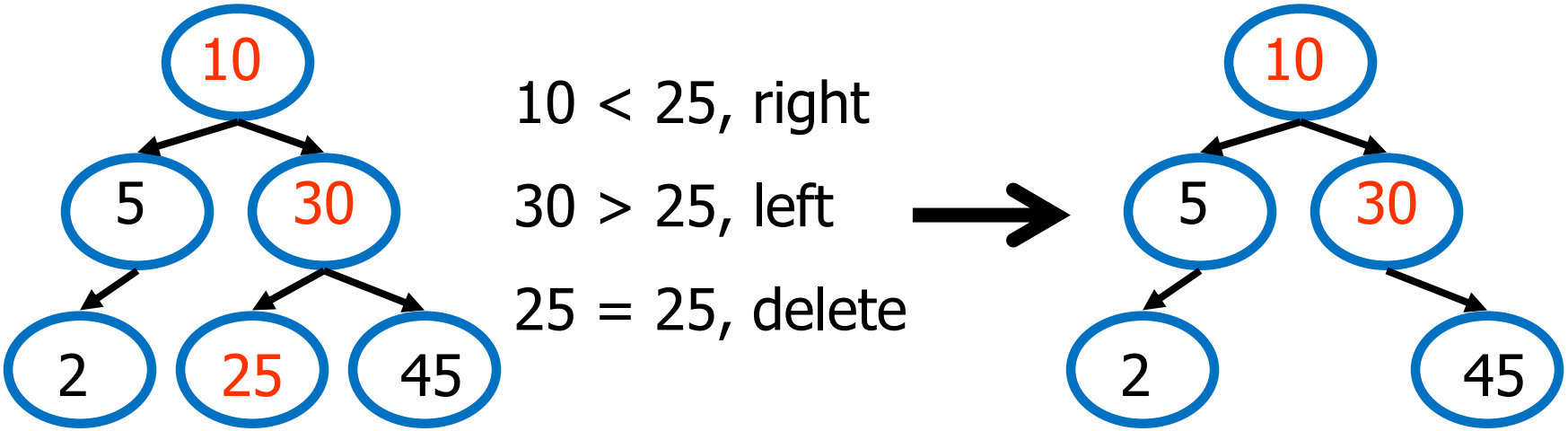
- A node being erased is not always going to be a leaf node
- There are three possible scenarios:
  - The node is a leaf node,
  - It has exactly one child, or
  - It has two children (it is a full node)



Deleting a Node – Leaf Node

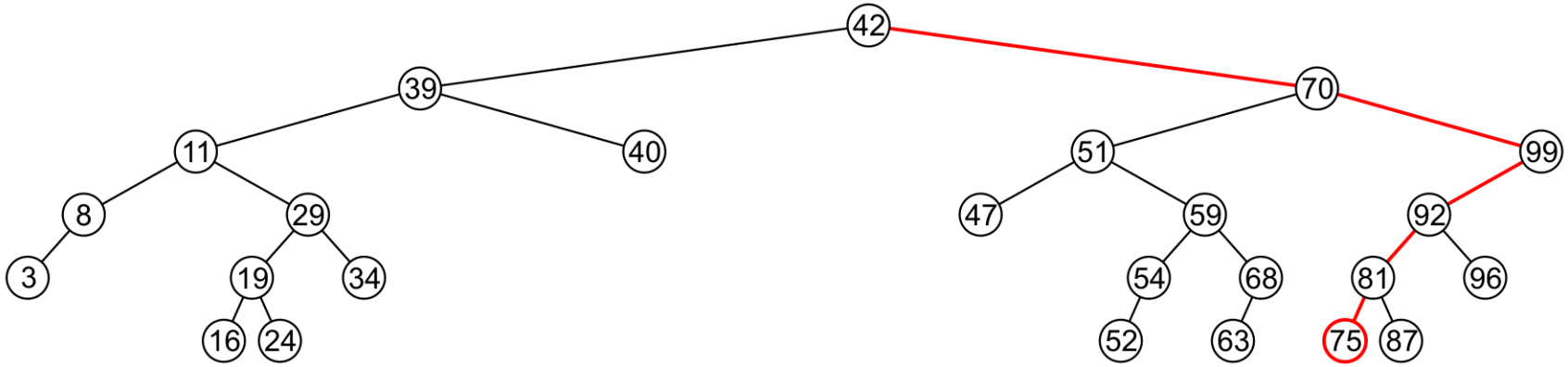
- Deleting a leaf node is easy
  - Find its parent
  - Set the child pointer that links to it to NULL
  - Free the node's memory

- Consider deleting node containing 25



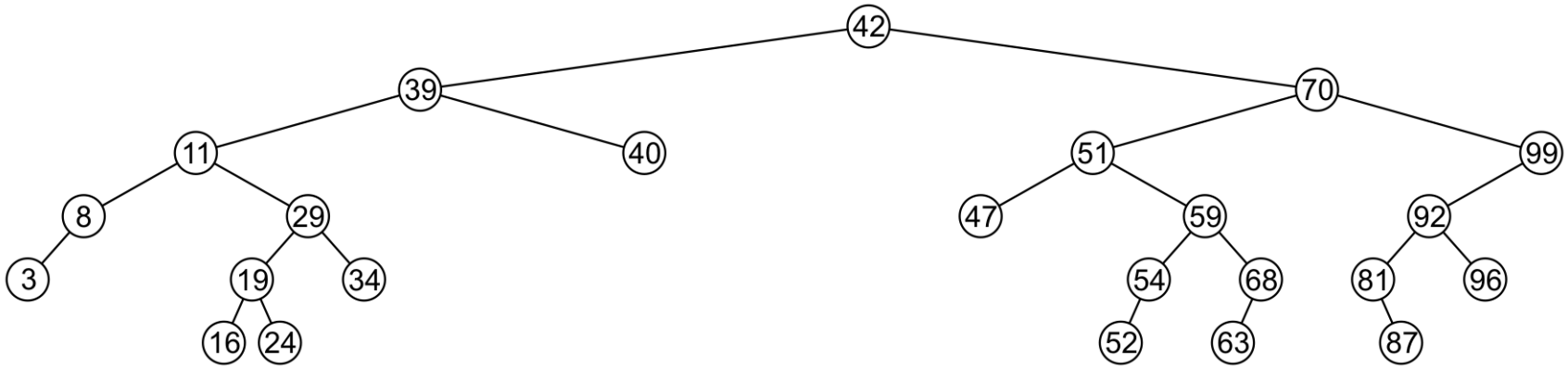
Deleting a Node – Leaf Node

- Consider deleting node containing 75



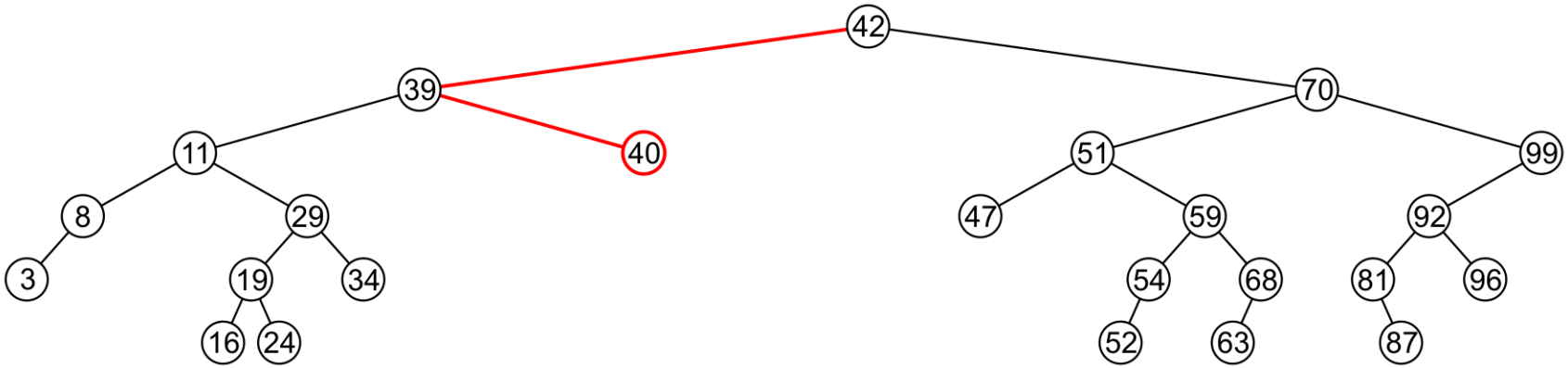
Deleting a Node – Leaf Node

- Consider deleting node containing 75
  - The node is deleted and left child of 81 is set to NULL



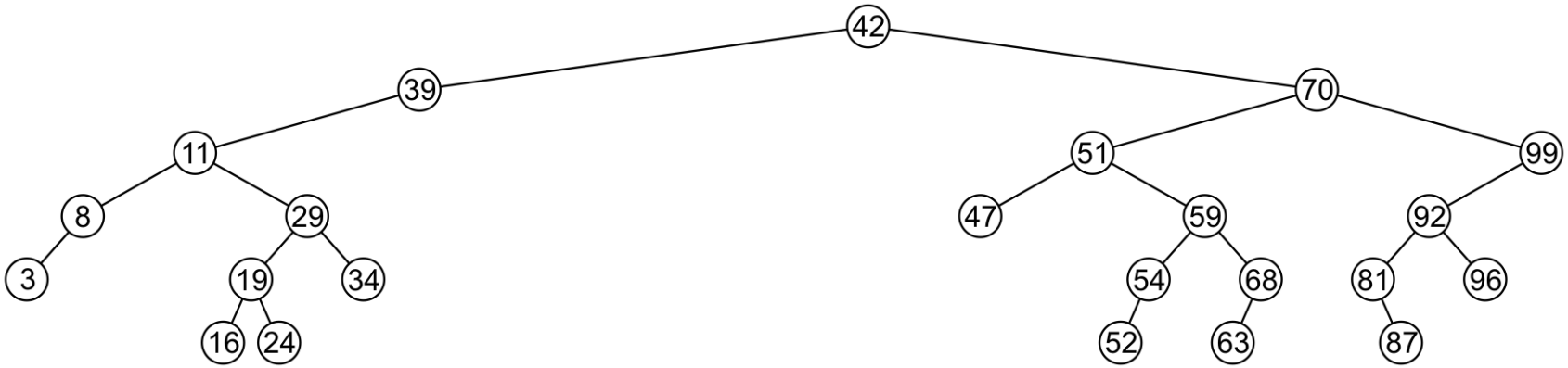
Deleting a Node – Leaf Node

- Consider deleting node containing 40



Deleting a Node – Leaf Node

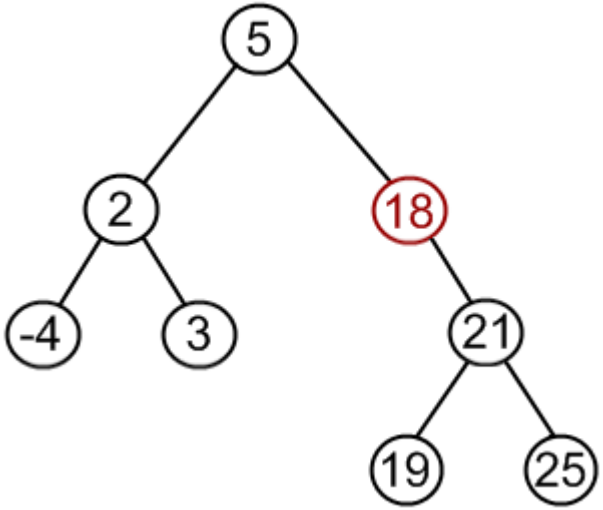
- Consider deleting node containing 40
  - Node is deleted and right child of 39 is set to NULL

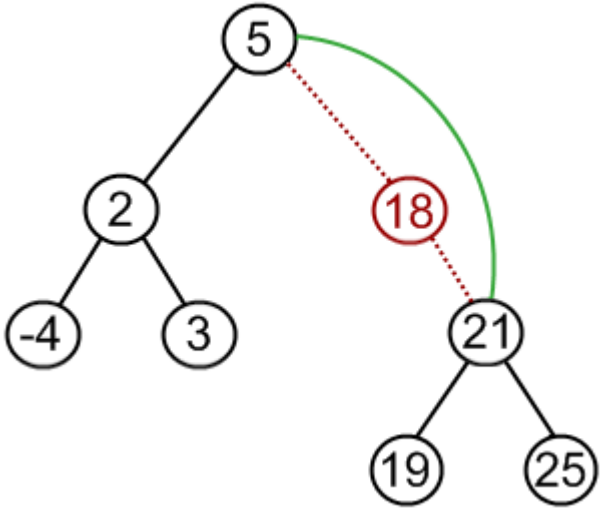


Deleting a Node – Node With Child

- If a node has only one child (left or right)
  - Simply promote the subtree associated with the child

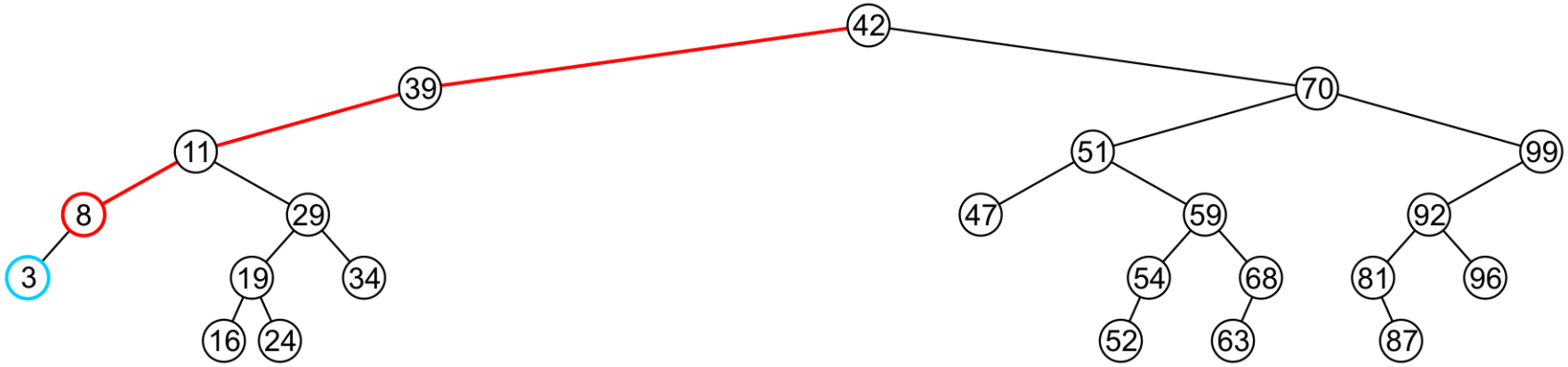
- Consider deleting 18 which has one right child
  - Node 18 is deleted and right tree of node 5 is update to point to 21





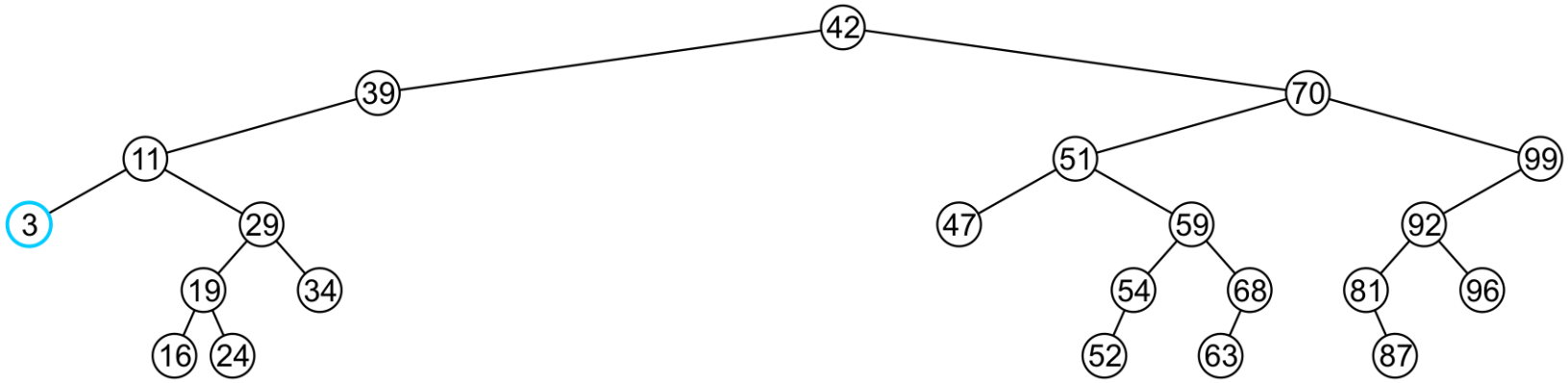
Deleting a Node – Node With Child

- Consider deleting 8 which has one left child



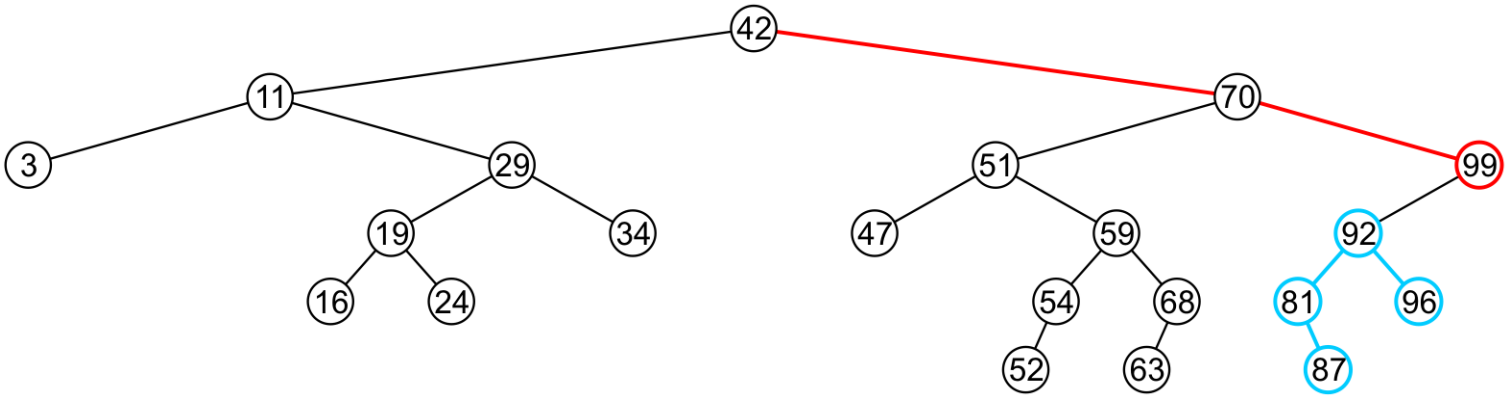
Deleting a Node – Node With Child

- Consider deleting 8 which has one left child
  - Node 8 is deleted and the left tree of 11 is updated to point to 3



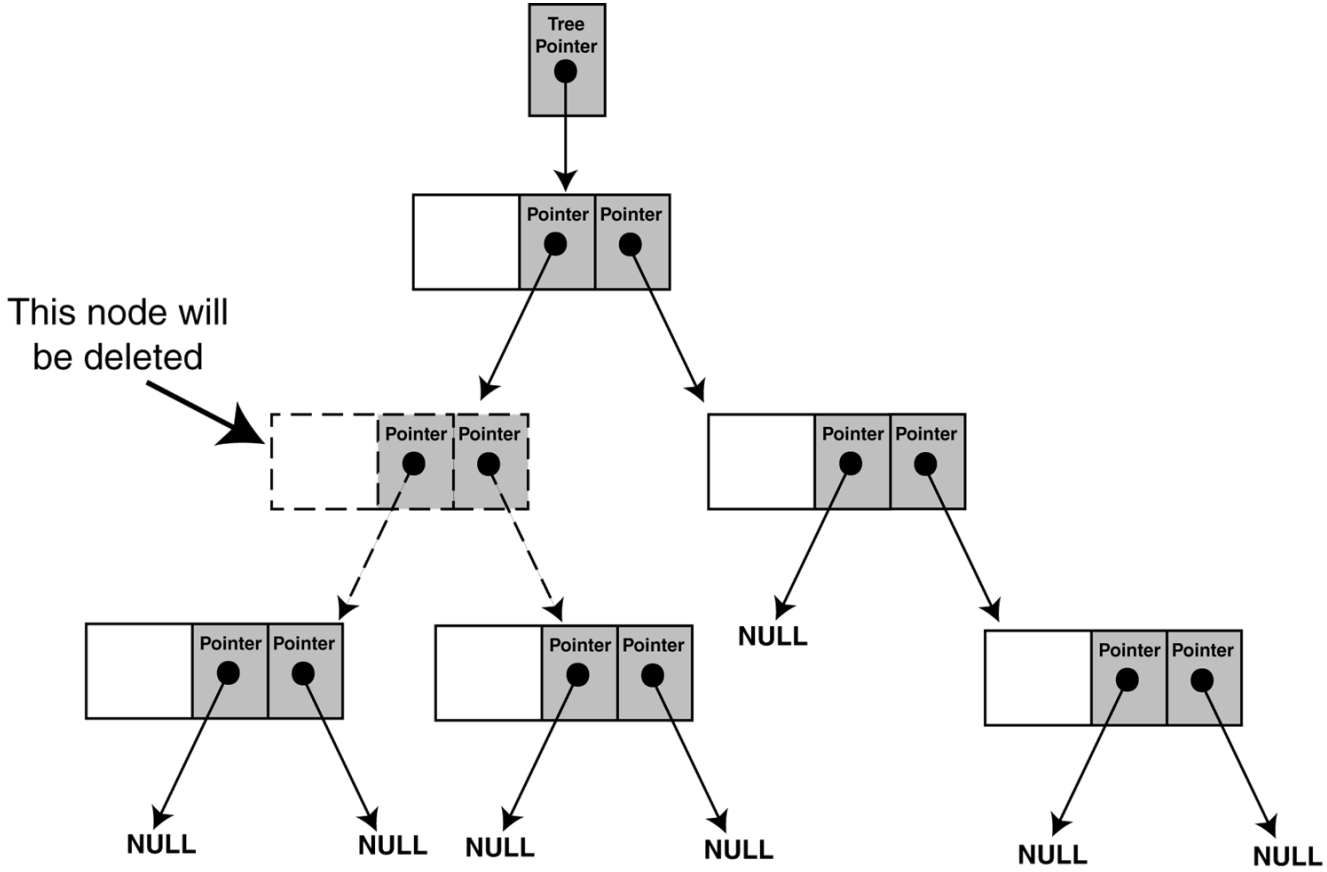
Deleting a Node – Node With Child

- Consider deleting the node containing 99



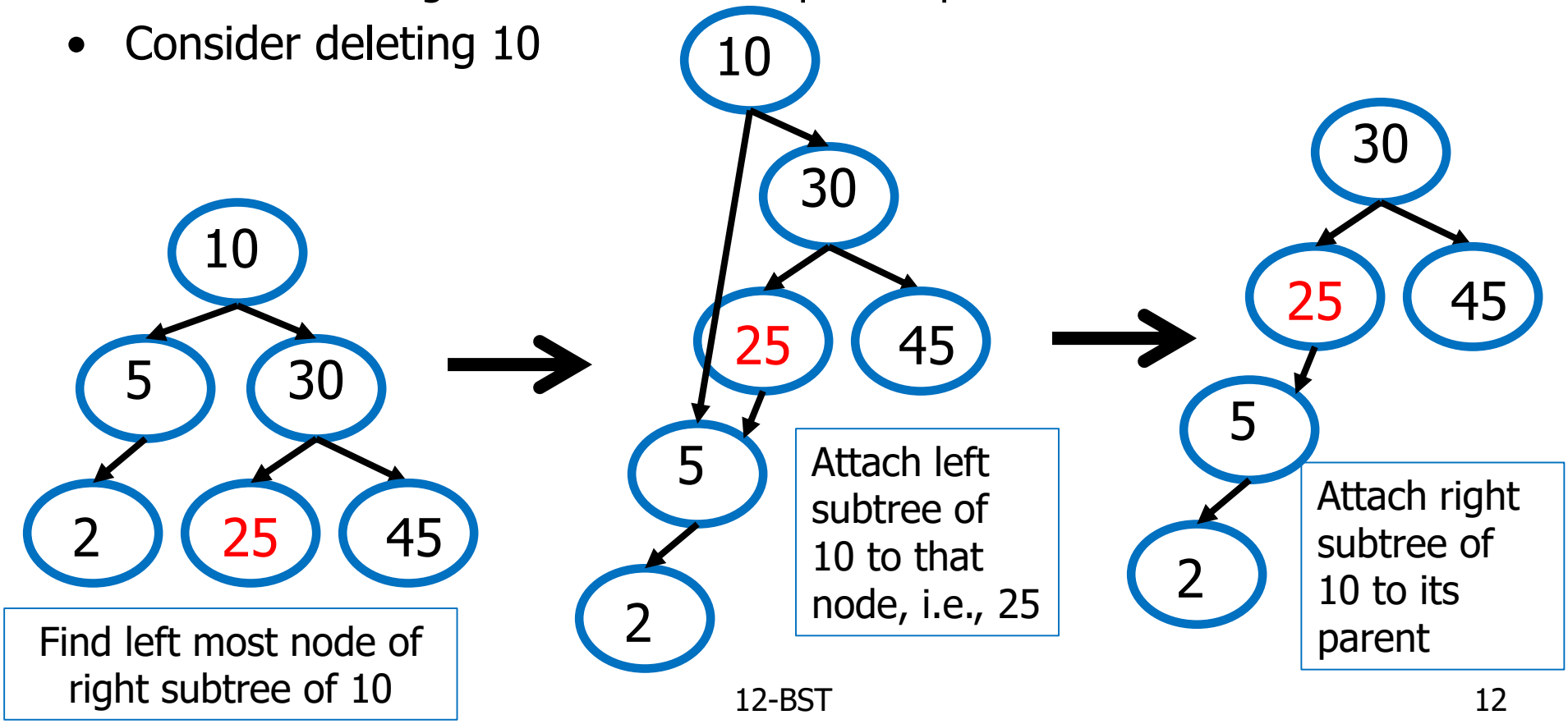
Deleting a Node – Node With Two Children

- The problem is not as easily solved if the node has two children



Deleting a Node – Node With Two Children

- Suppose node p with two children has to be deleted
  - Find a position in the right subtree of p to attach its left subtree
    - Left most node in the right subtree of node p
  - Attach the right subtree of node p to its parent
- Consider deleting 10



Pointers Review

- Pointer to Pointer versus reference to Pointer?

```
int g_One=1;

void func(int* pInt);

int main()
{
  int nvar=2;
  int* pvar=&nvar;
  func(pvar);
  std::cout<<*pvar<<std::endl;
  return 0;
}

void func(int* pInt)
{
  pInt=&g_One;
}
```

```
int g_One=1;

void func(int*& rpInt);

int main()
{
  int nvar=2;
  int* pvar=&nvar;
  func(pvar);
  std::cout<<*pvar<<std::endl;
  return 0;
}

void func(int*& rpInt)
{
  rpInt=&g_One;
}
```

Deleting a Node – Implementation

```
class IntBinaryTree {
   private:
      TreeNode *root;  // Pointer to the root of BST

      void destroySubTree(TreeNode *); //Recursively delete all tree nodes
      void deleteNode(int, TreeNode *&);
      void makeDeletion(TreeNode *&);
      void displayInOrder(TreeNode *);
      void displayPreOrder(TreeNode *);
      void displayPostOrder(TreeNode *);

   public:
      IntBinaryTree()              { root = NULL; }
      ~IntBinaryTree()             { destroySubTree(root); }
      void insertNode(int);
      bool find(int);
      void remove(int num);        { deleteNode( num, root)}
      void showNodesInOrder()      { displayInOrder(root);   }
      void showNodesPreOrder()     { displayPreOrder(root);  }
      void showNodesPostOrder()    { displayPostOrder(root); }
};
```

The argument passed to the remove function is the value of the node to be deleted.

Deleting a Node – Implementation

```
void IntBinaryTree::deleteNode(int num, TreeNode *&nodePtr)
{
    if (nodePtr == NULL) // node does not exist in the tree
        cout << num <<“ not found.\n";
    else if (num < nodePtr->value)
        deleteNode(num, nodePtr->left);  // find in left subtree
    else if (num > nodePtr->value)
        deleteNode(num, nodePtr->right); // find in right subtree
    else // num == nodePtr->value i.e. node is found
        makeDeletion(nodePtr); // actually deletes node from BST
}
```

Note:

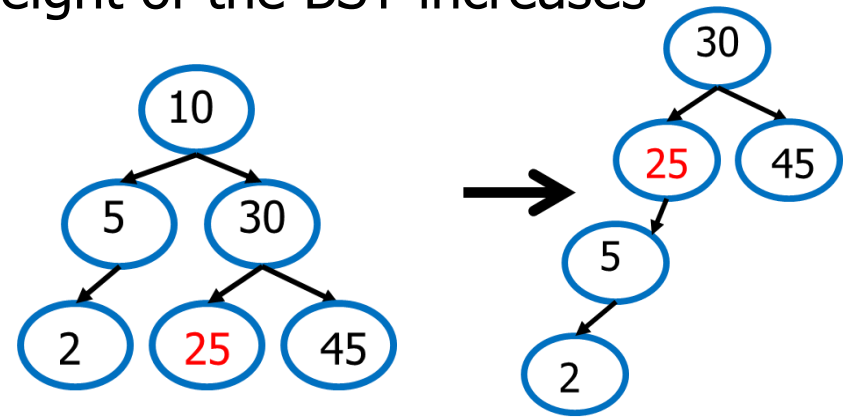
- The declaration of the `nodePtr` parameter: `TreeNode *&nodePtr;`
- `nodePtr` is a reference to a pointer to a `TreeNode` structure
  - Any action performed on `nodePtr` is actually performed on the argument passed into `nodePtr`

Deleting a Node – Implementation

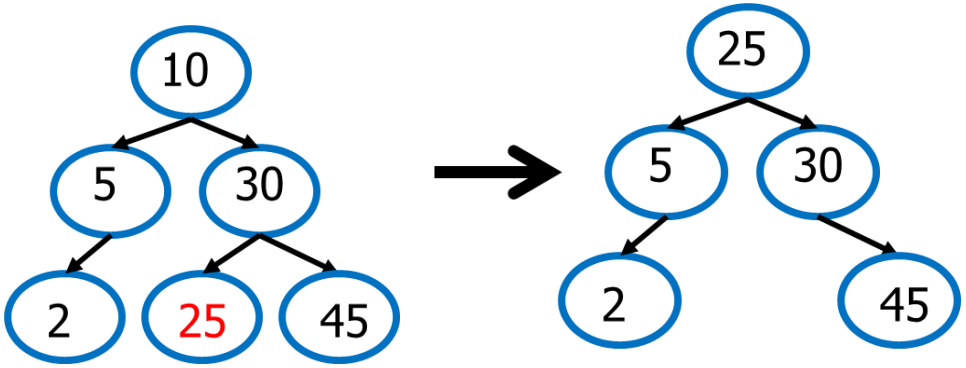
```
void IntBinaryTree::makeDeletion(TreeNode *&nodePtr) {
   TreeNode *tempNodePtr; // Temperary pointer
   if (nodePtr->right == NULL) { // case for leaf and one (left) child
      tempNodePtr = nodePtr;
      nodePtr = nodePtr->left; // Reattach the left child
      delete tempNodePtr;
   }
   else if (nodePtr->left == NULL) { // case for one (right) child
      tempNodePtr = nodePtr;
      nodePtr = nodePtr->right; // Reattach the right child
      delete tempNodePtr;
   }
   else {  // case for two children.
      tempNodePtr = nodePtr->right; // Move one node to the right
      while (tempNodePtr->left) { // Go to the extreme left node
         tempNodePtr = tempNodePtr->left;
      }
      tempNodePtr->left = nodePtr->left; // Reattach the left subtree
      tempNodePtr = nodePtr;
      nodePtr = nodePtr->right; // Reattach the right subtree
      delete tempNodePtr;
   }
}
```

Deleting a Node – Node With Two Children

- Problem: Height of the BST increases

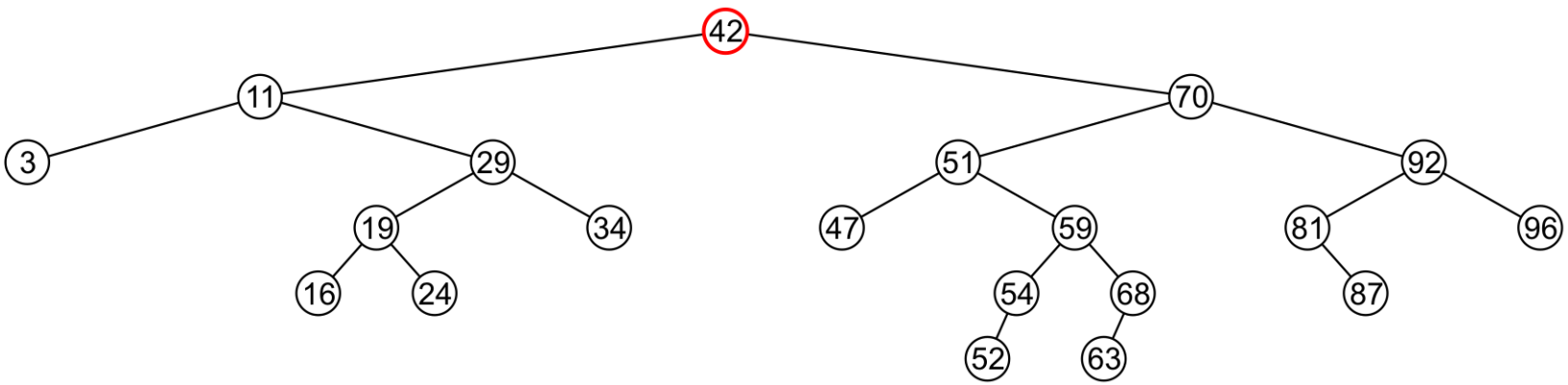


- A better Solution to delete node p with two children
  - Replace node p with the minimum object in the right subtree
  - Delete that object from the right subtree



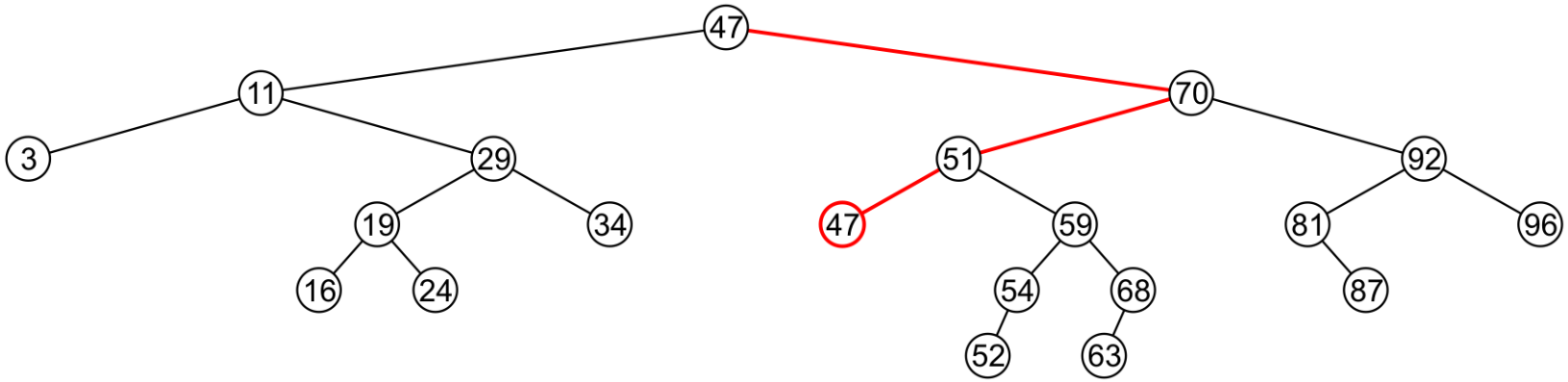
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 42



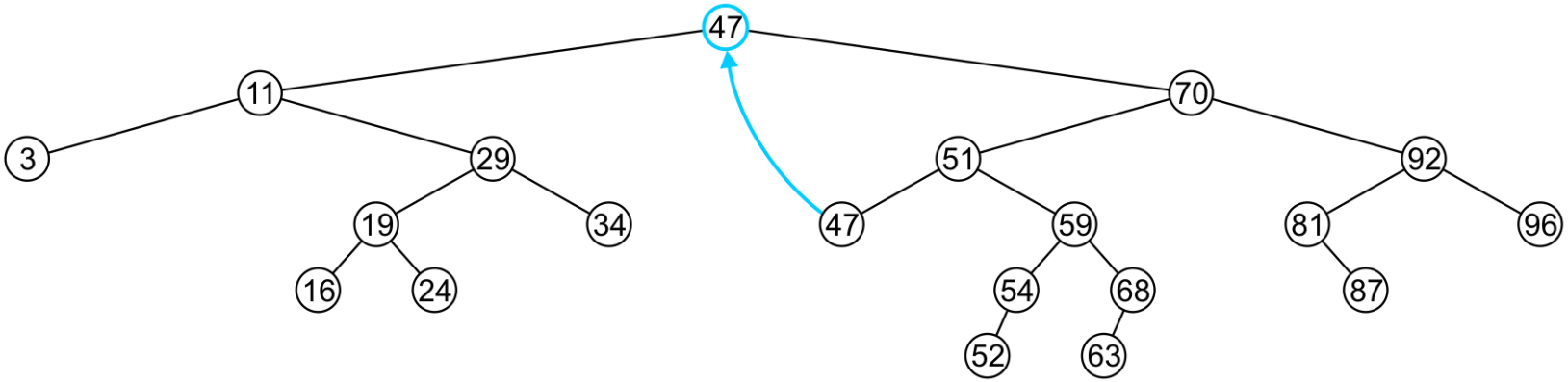
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 42
  - Find minimum object in the right subtree, i.e., 47



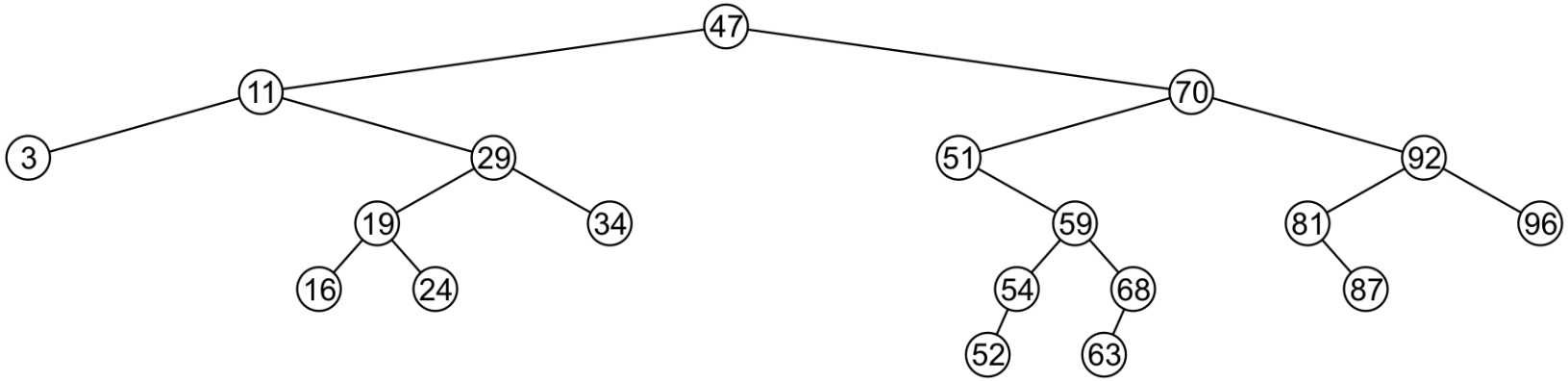
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 42
  - Find minimum object in the right subtree, i.e., 47
  - Replace 42 with 47



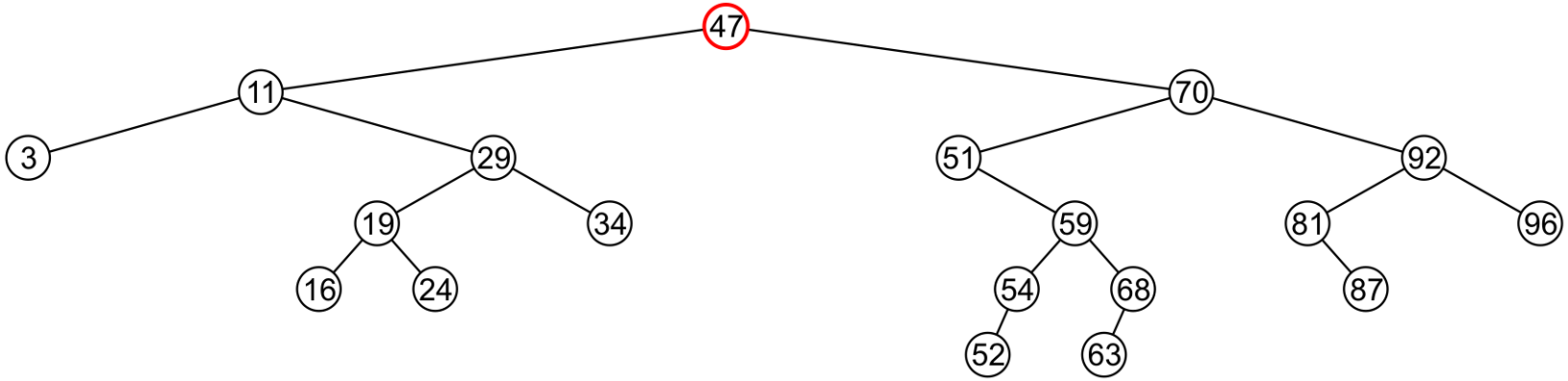
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 42
  - Find minimum object in the right subtree, i.e., 47
  - Replace 42 with 47
  - Delete the leaf node 47



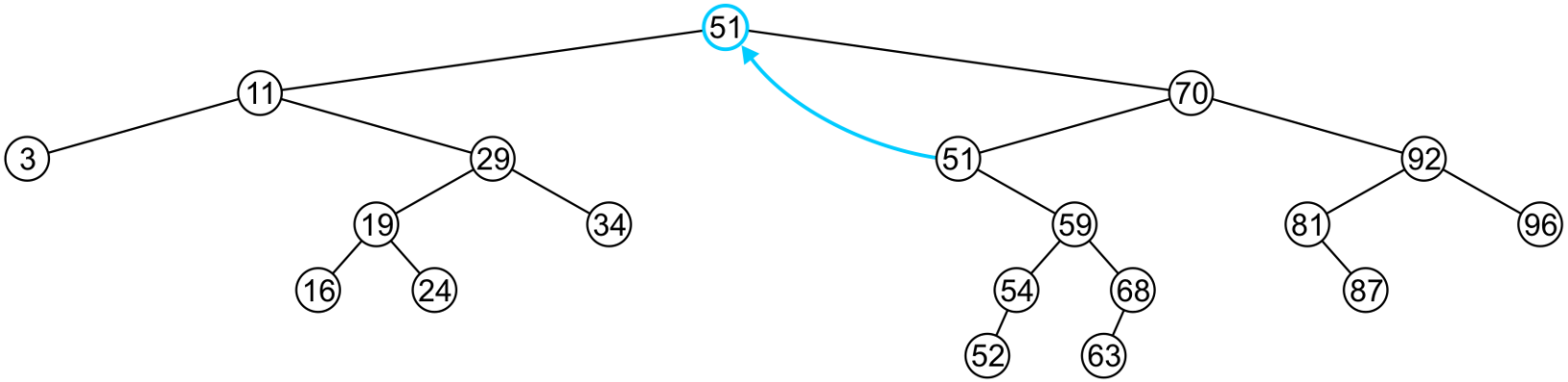
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 47



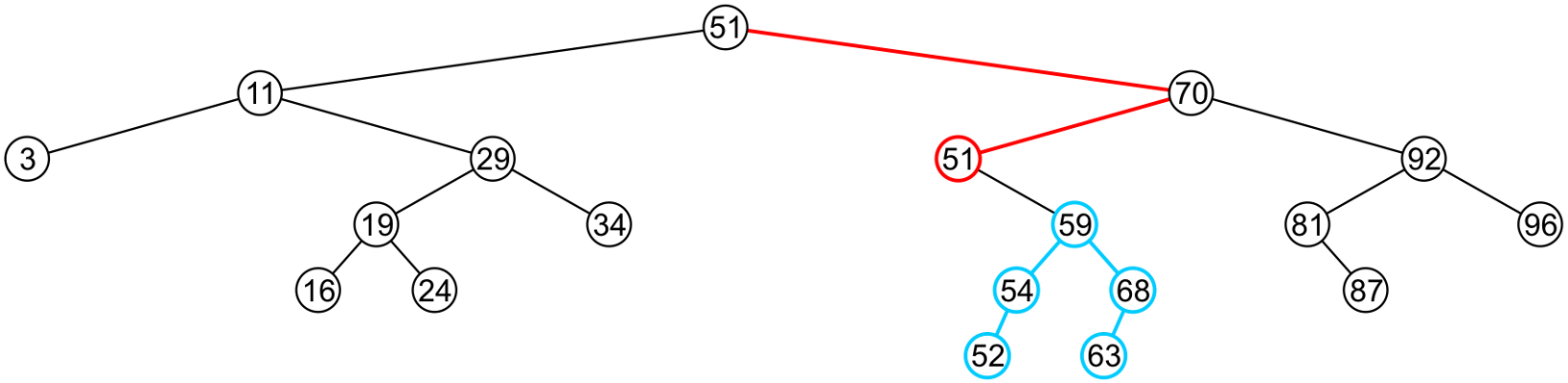
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 47
  - Replace 47 with 51



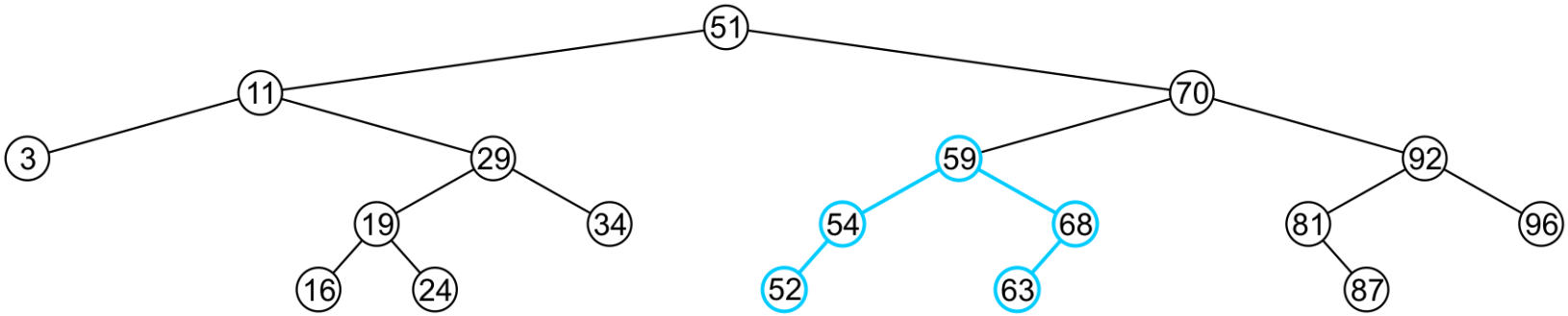
Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 47
  - Replace 47 with 51
  - Node 51 is not a leaf node



Deleting a Node Two Children – Better Solution

- Consider the problem of deleting a full node, e.g., 47
  - Replace 47 with 51
  - Node 51 is not a leaf node
    - Assign the left subtree of 70 to point to 59



Using BST

```
// This program builds a binary tree with 5 nodes.
// The DeleteNode function is used to remove two of them.
#include <iostream.h>
#include "IntBinaryTree.h“

void main(void) {
   IntBinaryTree tree;

   cout << "Inserting nodes.\n";
   tree.insertNode(5);
   tree.insertNode(8);
   tree.insertNode(3);
   tree.insertNode(12);
   tree.insertNode(9);

   cout << "Here are the values in the tree:\n";
   tree.showNodesInOrder();
   cout << "Deleting 8...\n";
   tree.remove(8);
   cout << "Deleting 12...\n";
   tree.remove(12);

   cout << "Now, here are the nodes:\n";
   tree.showNodesInOrder();
}
```

Program Output:
Inserting nodes.
Here are the values in the tree:
3
5
8
9
12

Using BST

```
// This program builds a binary tree with 5 nodes.
// The DeleteNode function is used to remove two of them.
#include <iostream.h>
#include "IntBinaryTree.h“

void main(void) {
   IntBinaryTree tree;

   cout << "Inserting nodes.\n";
   tree.insertNode(5);
   tree.insertNode(8);
   tree.insertNode(3);
   tree.insertNode(12);
   tree.insertNode(9);

   cout << "Here are the values in the tree:\n";
   tree.showNodesInOrder();
   cout << "Deleting 8...\n";
   tree.remove(8);
   cout << "Deleting 12...\n";
   tree.remove(12);

   cout << "Now, here are the nodes:\n";
   tree.showNodesInOrder();
}
```

Program Output:

```
Inserting nodes.
Here are the values in
the tree:
3
5
8
9
12
Deleting 8...
Deleting 12...
Now, here are the
nodes:
3
5
9
```

Traversing a Binary Search Tree

```
class IntBinaryTree {
   private:
      TreeNode *root;  // Pointer to the root of BST

      void destroySubTree(TreeNode *); //Recursively delete all tree nodes
      void deleteNode(int, TreeNode *&);
      void makeDeletion(TreeNode *&);
      void displayInOrder(TreeNode *);
      void displayPreOrder(TreeNode *);
      void displayPostOrder(TreeNode *);

   public:
      IntBinaryTree()                 { root = NULL; }
      ~IntBinaryTree()                { destroySubTree(root); }
      void insertNode(int);
      bool find(int);
      void remove(int);               { deleteNode( num, root); }
      void showNodesInOrder()         { displayInOrder(root);   }
      void showNodesPreOrder()        { displayPreOrder(root);  }
      void showNodesPostOrder()       { displayPostOrder(root); }
};
```

Recursive implementation as discussed in the slides of Tree Traversal chapter.

Using BST

```
// This program builds a binary tree with 5 nodes.
// The nodes are displayed with inorder, preorder, and post
#include <iostream.h>
#include "IntBinaryTree.h“

void main(void)
{
   IntBinaryTree tree;

   cout << "Inserting nodes.\n";
   tree.insertNode(5);
   tree.insertNode(8);
   tree.insertNode(3);
   tree.insertNode(12);
   tree.insertNode(9);


   cout << "Inorder traversal:\n";
   tree.showNodesInOrder();
   cout << "\nPreorder traversal:\n";
   tree.showNodesPreOrder();
   cout << "\nPostorder traversal:\n";
   tree.showNodesPostOrder();
}
```

Program output:

```
Inserting nodes.
Inorder traversal:
3
5
8
9
12

Preorder traversal:
5
3
8
12
9


Postorder traversal:
3

9
12
8
5
```

BST's Practice

While inserting the elements 71, 65, 84, 69, 67, 83 in an empty binary search tree (BST) in the sequence shown, the element in the lowest level is

A 65

B 67

C 69

D 83

BST's Practice

Suppose the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the usual ordering on natural numbers. What is the in-order traversal sequence of the resultant tree?

A 7 5 1 0 3 2 4 6 8 9

B 0 2 4 3 1 6 5 9 8 7

C 0 1 2 3 4 5 6 7 8 9

D 9 8 6 4 2 3 0 1 5 7

The following numbers are inserted into an empty binary search tree in the given order: 10, 1, 3, 5, 15, 12, 16. What is the height of the binary search tree (the height is the maximum distance of a leaf node from the root)? (GATE CS 2004)

A 2

B 3

C 4

D 6

Constructing a Binary Search Tree Given its pre-order traversal

The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the postorder traversal sequence of the same tree?

(A) 10, 20, 15, 23, 25, 35, 42, 39, 30

(B) 15, 10, 25, 23, 20, 42, 35, 39, 30

(C) 15, 20, 10, 23, 25, 42, 35, 39, 30

(D) 15, 10, 23, 25, 20, 35, 42, 39, 30

Constructing a Binary Search Tree Given its postorder traversal

- Construct a binary tree given its postorder traversal:
- {1, 7, 5, 50, 40, 10}

- Answer to previous question will be:

```
      10
    /   \
   5     40
  / \      \
 1   7      50
```

Can a specific tree be constructed from inorder traversal?

Which of the following traversals is sufficient to construct BST from given traversals 1) Inorder 2) Preorder 3) Postorder

A. Any one of the given three traversals is sufficient

B. Either 2 or 3 is sufficient

C. 2 and 3

D. 1 and 3

Any Question So Far?